# Experimentos com um MLP implementado do zero

CI1026 - Visão Computacional e Percepção — TA05 código: https://github.com/AlmiranteFuchs/TA05-mlp

## 1. Introdução

A tarefa: estudar o notebook de referência "MLP from scratch" (adaptado de um notebook do Kaggle pelo prof.), ajudar a depurar, e descobrir como classificar bem as três classes do Iris — analisando convergência, erros e overfitting. O Iris tem 150 amostras de 4 features (medidas de sépala e pétala) e 3 classes; a rede da referência é um 4-5-3 com sigmoid, MSE e SGD por amostra com one-hot na saída.

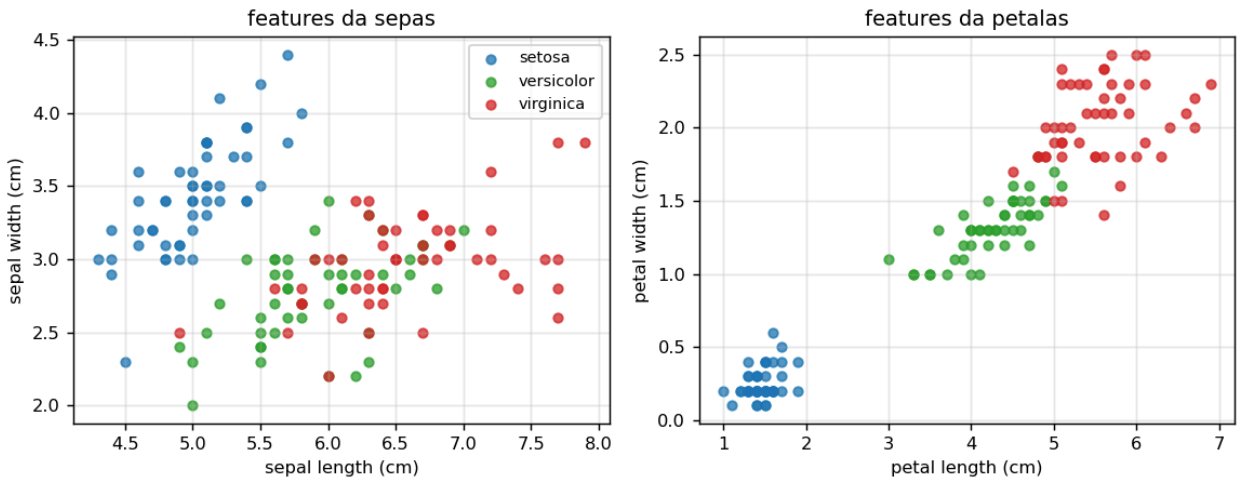



Figura 1: as 3 classes do Iris: sepas (esq.) quase inseparaveis, petalas (dir.) bem melhores

O spoiler do relatório inteiro: **a rede de referência aprende direitinho, mas o `predict` dela joga o aprendizado fora** — treinada por 3000 épocas ela marca 33% de acurácia no teste (chute), enquanto a *mesma rede, nos mesmos pesos*, com o predict correto marca 100%. Detalhes abaixo.

## 2. Depuração do código de referência

Portei o notebook fielmente (`src/mlp_ref.py`, bugs preservados) e uma versão corrigida (`src/mlp.py`). O que encontrei:

- **`predict` não aplica a função de ativação.** O forward de treino usa sigmoid nas duas camadas; o predict faz só `X @ W1 + b1 @ W2 + b2`, um forward linear. Como as ativações *escondidas* também são descartadas, a função decidida muda de verdade — não é só o argmax de saída reescrito. Resultado: **acc de teste = 0.333**; trocando só o predict pelo forward com ativação, a mesma rede treinada dá **1.000**. A rede aprendia; quem reprovava era o predict.
- **Bias de saída atualizado dentro do loop dos neurônios escondidos.** Em `Backpropagation_Algorithm`, `BIAS_output[j] -= lr*delta` fica dentro do `for i in range(hiddenLayer)`: o bias de saída andava 5x mais rápido que o planejado (e recebia o mesmo update 5 vezes).
- **Biases criados como inteiros.** `np.array([-1,-1,-1])` é int64; o `arr[j] -= 0.4` por elemento trunca para inteiro em vez de acumular — na prática os biases nunca saem de -1 durante o treino. Bug silencioso: não

dá erro nenhum, só não aprende o bias. (Na minha versão, `np.full` com dtype=float — sem o fix, o NumPy novo inclusive levanta `UFuncTypeError` na versão vetorizada.)

- **O "erro" reportado não é o erro.** `total_error += square_error` fica dentro do loop dos 3 neurônios de saída, somando somas parciais (o erro do neurônio 0 conta 3x, o do 1 conta 2x), com fator 0.05 em vez de 0.5. O número que o fit imprime convergindo para ~0.005 é ~4x menor que o MSE real (~0.021 no fim) — a curva de erro do notebook dá uma confiança que o modelo não tem.
- **`params=None` quebra o construtor.** O default usa `iasHiddenValue` (typo) e depois lê `self.BiasHiddenValue` → `AttributeError` na cara.
- **Quirk menor:** o erro da camada escondida é propagado com `WEIGHT_output` *já atualizado* (o textbook usa os pesos antigos — convergência quase igual na prática, mas não é o gradiente exato).
- **Faltam:** seed/validação/shuffle por época — tudo o que a análise de convergência e overfitting pede. E `classes_number` fixo em 3.

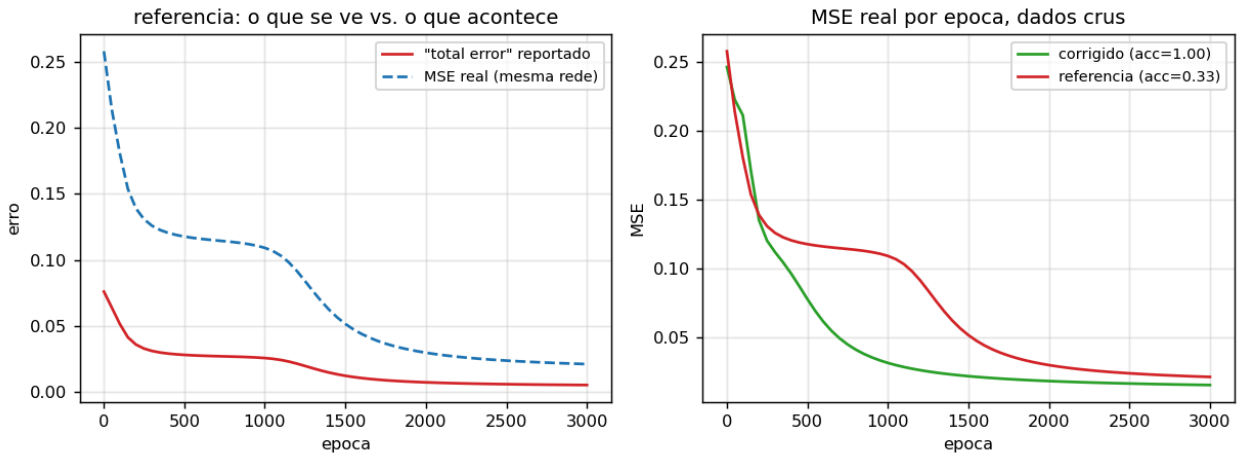



Figura 2: esq.: o 'erro' que o codigo de referencia mostra vs. o MSE real da mesma rede; dir.: MSE real da referencia vs. versao corrigida (dados crus)

À esquerda da figura: o erro que o código mostra vs. o MSE real da mesma rede — a curva "bonita" esconde um MSE 4x maior. À direita: MSE real da referência vs. a versão corrigida, mesmos dados crus — a corrigida converge mais cedo (o bias de saída e o gradiente exato ajudam), e é ela que entrega 100% de teste porque o predict agora concorda com o treino.

## 3. Setup dos experimentos

Split estratificado 80/20 igual ao do notebook (`random_state=11`): 120 treino, 30 teste. Para a análise de overfitting, um split interno 96 treino / 24 validação. Normalização z-score ajustada só no treino. Todos os sweeps com 3 seeds; média ± desvio no teste.

## 4. Convergência

**Normalização.** Com as features cruas (pétala ~1-7 cm), a sigmoid trabalha saturada e a descida é lenta — foram as ~1500 primeiras épocas só para "destravar". Padronizando as features, a mesma rede e os mesmos pesos iniciais caem para MSE ~0.01 em menos de 400 épocas. Acc final igual (1.0 nos dois casos, a tarefa é fácil), mas a velocidade muda tudo na prática.

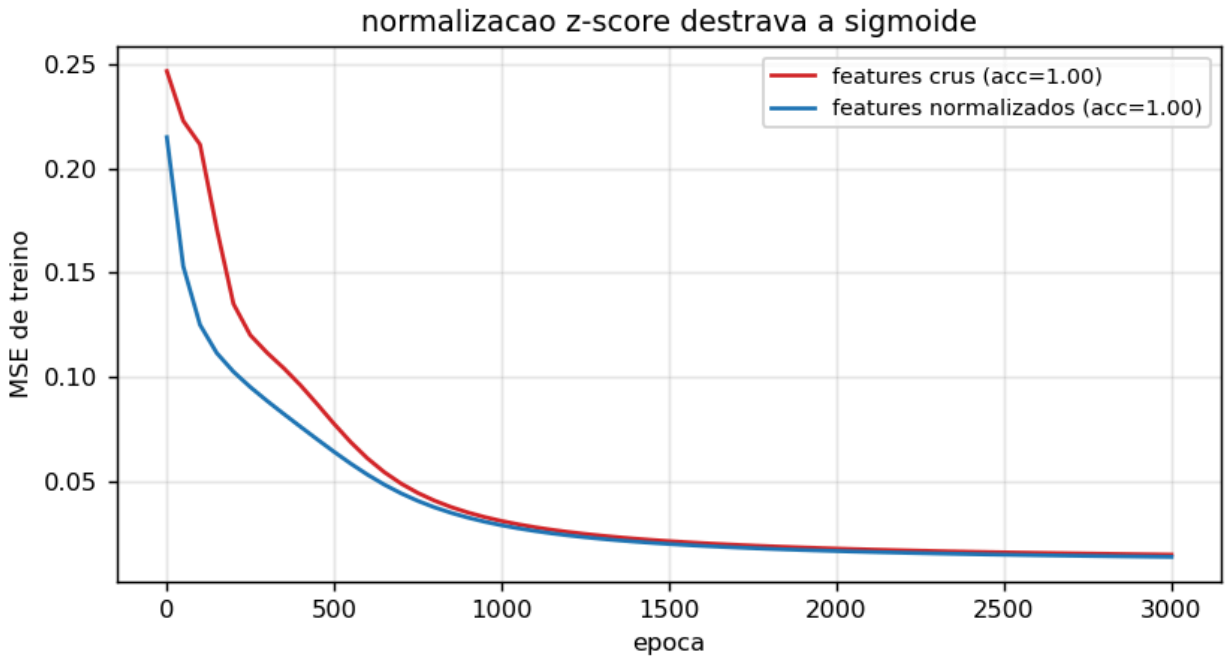



Figura 3: normalizacao z-score: mesma rede, mesmos pesos iniciais — convergencia muito mais rapida

**Learning rate.** Sweep de 0.0005 a 2.0 (rede 4-5-3, sigmoid, dados normalizados, 1200 épocas):

| lr | acc teste (3 seeds) |
|---|---|
| 0.0005 | 0.844 ± 0.031 |
| 0.001 | 0.900 ± 0.072 |
| 0.005 | 1.000 ± 0.000 |
| 0.01–2.0 | 1.000 ± 0.000 |

O lr=0.005 do notebook funciona, mas é conservador: a janela útil aqui vai de ~0.005 a 2.0 — MSE + sigmoid em dados normalizados é bem comportado, e nem lr=2.0 diverge (só oscila mais no começo). Abaixo de ~0.005 a rede simplesmente não chega lá em 1200 épocas.

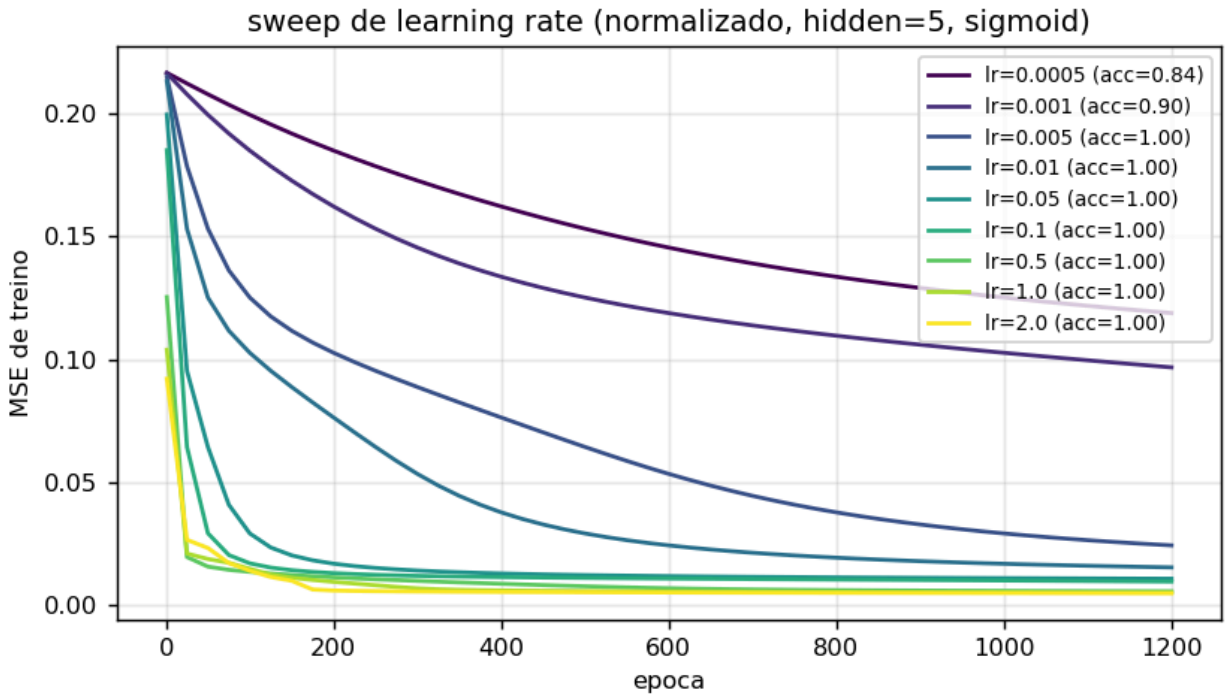



Figura 4: sweep de learning rate: curvas de MSE por epoca (acc de teste na legenda)

## 5. Capacidade e função de ativação

**Neurônios escondidos.** Com lr=0.05: H=1 já dá 0.989 ± 0.016; de H=2 para cima, 1.000 em todas as seeds. Faz sentido: versicolor×virginica são quase separáveis linearmente nas pétalas — a camada escondida precisa de pouquíssima capacidade para o Iris.

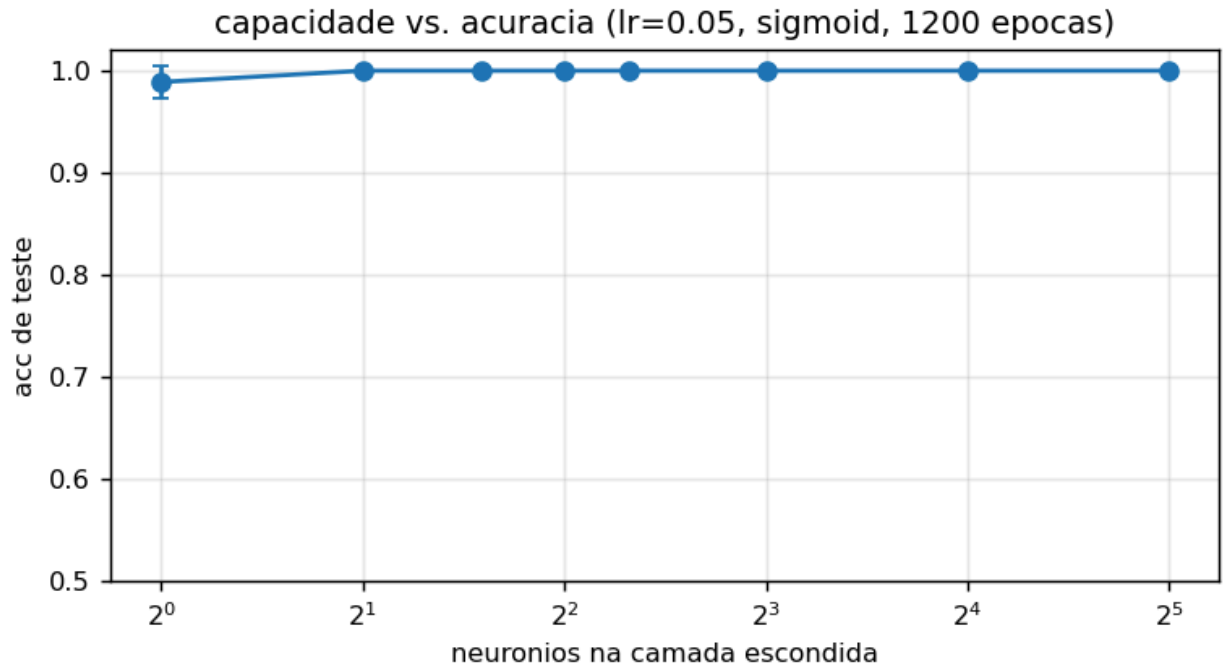



Figura 5: acuracia de teste vs. neuronios na camada escondida (media +- desvio, 3 seeds)

**Ativação.** Aqui veio a surpresa do dia (lr=0.05, H=5, bias=-1):

| ativação | acc teste (3 seeds) |
|---|---|
| sigmoid | 1.000 ± 0.000 |
| tanh | 1.000 ± 0.000 |
| relu (bias=-1) | 0.322 ± 0.016 |
| relu (bias=0.0) | 0.856 ± 0.140 |

ReLU com bias inicial -1 é uma **rede morta**: os pré-ativações nascem negativos, a saída dos neurônios trava em 0, o gradiente é 0 e nada aprende — a rede colapsa para prever uma classe só (~33%). Subindo o bias para 0 ela revive, mas continua instável (uma seed em 0.67): ReLU pede init melhor que "uniforme ±1 e bias constante". Para essa arquitetura/dataset, sigmoid e tanh são as escolhas seguras — e por isso o notebook original usa sigmoid.

## 6. Overfitting

**Com dados limpos, não tem overfitting para mostrar** — e isso é resultado. Treinando H=4 e H=64 por 2500 épocas, a validação acompanha o treino o tempo todo (val_acc bate 1.0 já na época ~25 e não sai de lá). Iris normalizado é separável demais: capacidade extra não degrada generalização, só desperdiça época — o early stopping na melhor val (~época 25-200) economiza ~90% do treino sem custar nada.

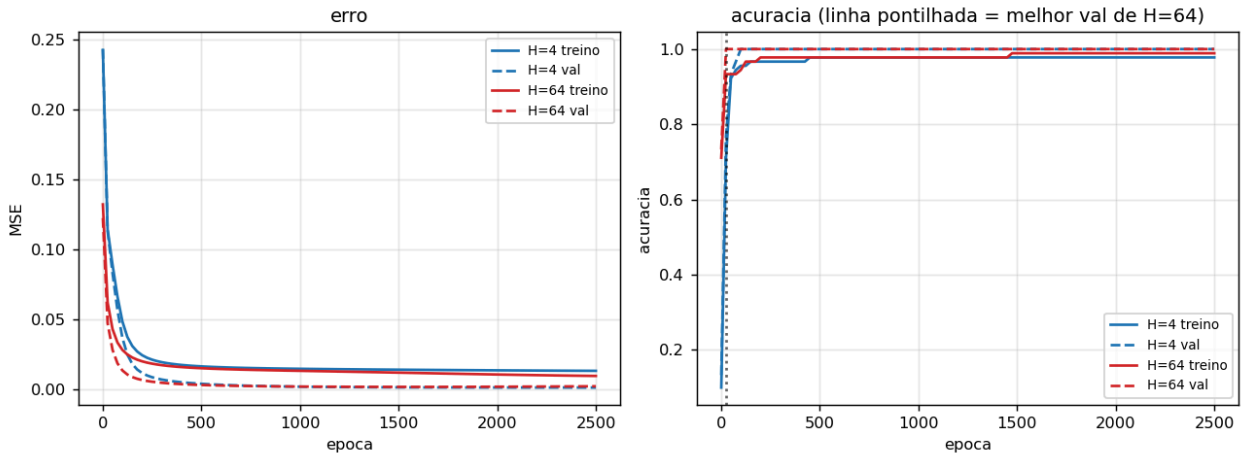



Figura 6: dados limpos: nem H=64 overfitta — val acompanha treino (Iris e separavel demais)

Para ver o fenômeno de verdade, injetei 20% de rótulos embaralhados no treino. Aí a assinatura clássica aparece no modelo grande: o H=64 vai **memorizando o ruído** — a acurácia de treino (contra os rótulos ruidosos) sobe até 0.92 enquanto a val cai de 1.00 para 0.97, e o `val_loss` faz fundo ~época 1000 e volta a subir enquanto o `train_loss` continua caindo. O H=4, sem capacidade para memorizar, segura a val em 1.0. Ou seja: gap treino-val crescente = overfitting; aqui o remédio é early stopping (melhor val na época 50) ou menos capacidade — o H=4 já resolve.

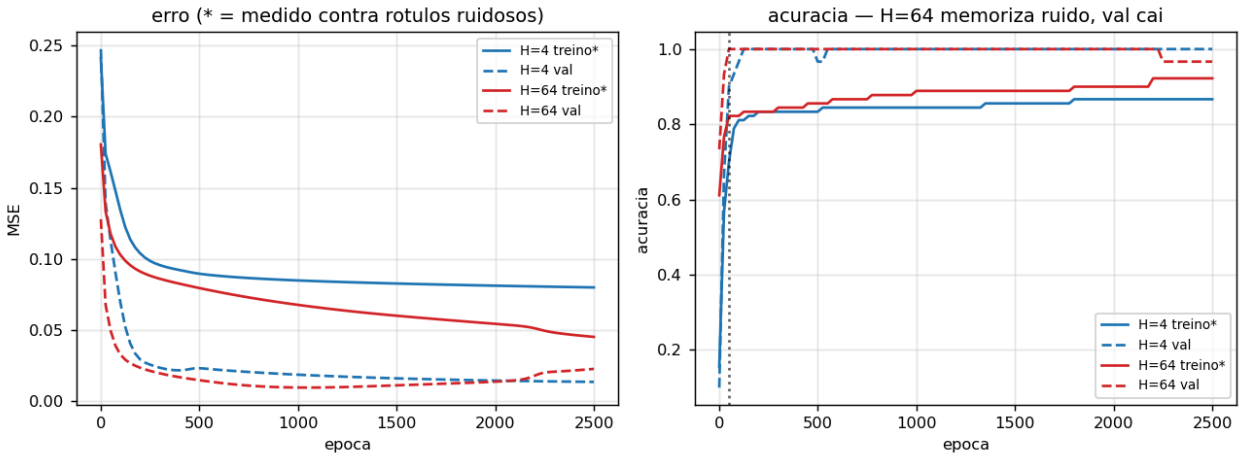



Figura 7: com 20% de ruido nos rotulos: H=64 memoriza o ruido (treino* sobe, val cai; val_loss diverge a partir de ~epoca 1000)

## 7. Como classificar bem as três classes

Juntando o que os experimentos mostraram, a receita:

1. **predict com ativação** (o bug nº1) — sem isso, nada importa;
2. **normalizar as features** (z-score) — destrava a sigmoid e encurta o treino em ~4x;
3. **sigmoid ou tanh**; relu só com init/bias melhores;
4. **lr ~0.05** (0.005-2.0 funcionam; o original 0.005 é só lento);
5. **H pequeno basta** — H=2 já dá 1.0; usei H=8 por margem;
6. **early stopping na validação** — ~200-800 épocas, não 3000.

Config final: `4 -> 8 -> 3`, sigmoid, lr=0.05, bias=-1, 800 épocas, dados normalizados → **100% no teste** (30/30), batendo com `sklearn.MLPClassifier(8)` que também dá 1.0. A matriz de confusão sai perfeita — nem versicolor×virginica escapam.

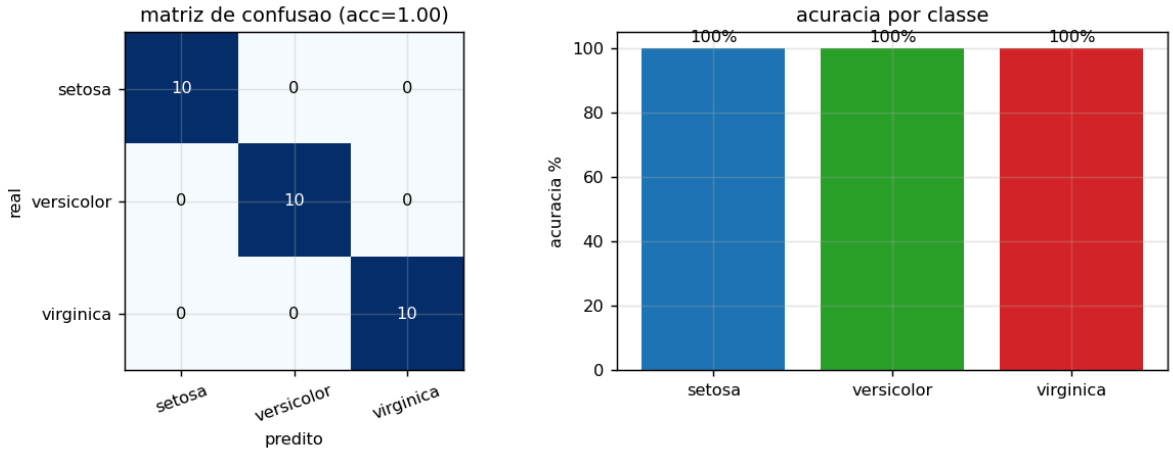



Figura 8: modelo final (4-8-3, sigmoid, lr=0.05): matriz de confusao e acuracia por classe

E como as pétalas carregam quase toda a informação, uma rede 2-8-3 treinada só em `petal length/width` também fecha 100% de teste — as regiões de decisão ficam quase lineares, com uma curva suave entre versicolor e virginica exatamente onde os pontos se encostam:

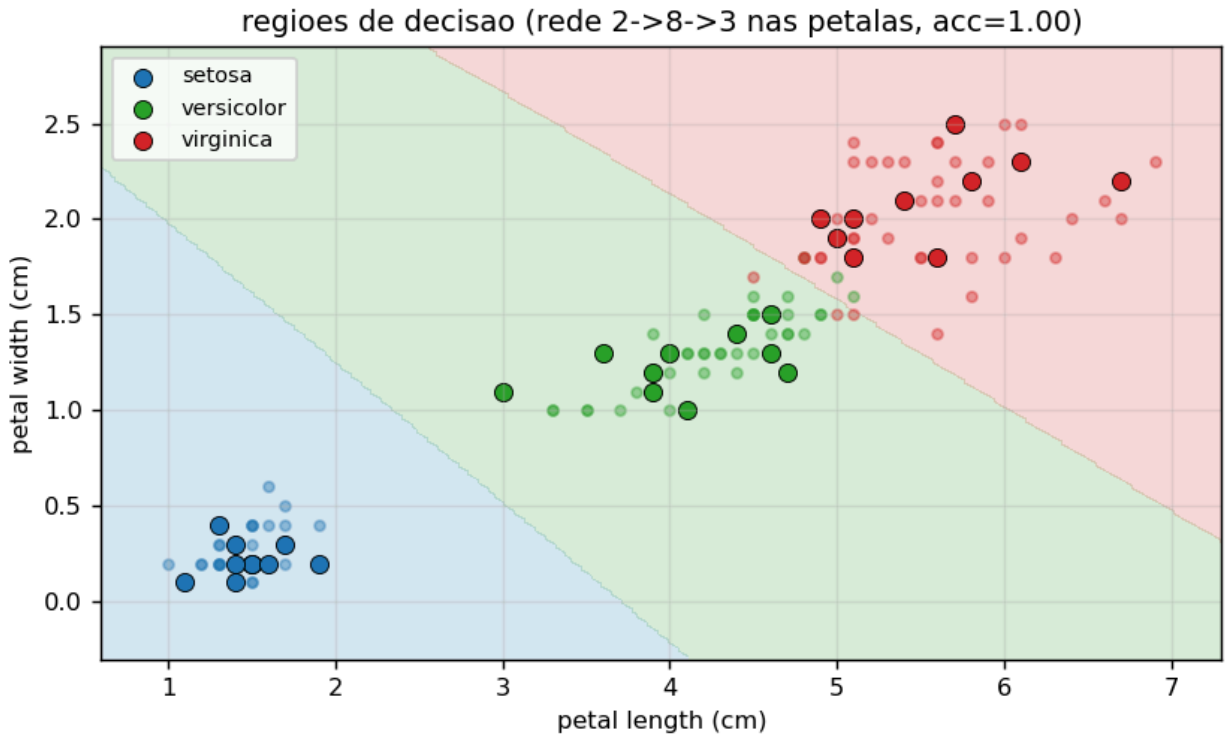



Figura 9: regioes de decisao de uma rede 2-8-3 treinada so nas features de petala (pontos grandes = teste, pequenos = treino)

## 8. Conclusão

A implementação de referência tinha a mecânica do backprop correta — os sinais de gradiente estão certos — mas cinco bugs em volta (predict linear, bias no loop errado, bias int truncado, métrica de erro furada, typo no construtor) faziam o notebook reportar uma convergência bonita para um modelo que erra 2/3 do teste. Depurado, o mesmo MLP com normalização e parada cedo acerta 100% das três classes — e a "dificuldade" da tarefa estava muito mais no código que no problema. O Iris só deixa ver overfitting quando se estraga os rótulos de propósito — aí o modelo grande memoriza o ruído na hora, e a curva de validação denuncia.

## Referências


- Notebook da disciplina: "Multi-Layer Perceptron from scratch" (adaptado de V. G. Lemos, Kaggle — link no Colab do AVA)
- Slides da disciplina: "Multi-layer Perceptron (MLP)" (Prof. Eduardo Todt)
- scikit-learn docs: `MLPClassifier`, `StandardScaler`, `load_iris`